

* for statement

• for loop is an entry-controlled loop

syntax: $\text{for}(\text{initialization}; \text{test Cond}^n; \text{increment/decrement})$
 $\{$
 $\underline{\hspace{1cm}}$
 $\}$

• initialization of Control variable is done first.

• The value of Control variable is tested using test condition. If the Condition is true loop is executed otherwise loop is terminated and execution continues with the statement that immediately follows the loop.

• When loop is executed the control is transferred back to the for statement after evaluating the last statement in the loop & control variable is incremented/decremented. This process continues till the value of the control variable fails to satisfy the test condition.

Print sum of 1st 5 no

```
void main()
```

```
{  
  int i, sum = 0;
```

```
  for (i = 1, sum = 0; i <= 5; i++)
```

```
{  
  sum = sum + i;
```

```
  printf("\n sum = %d", sum);  
}
```

Additional features of for loop

- More than one variable can be initialized at a time in the for statement.

```
for (p=1, n=0; n<17; ++n)
```

- More than one variable can be ~~initialized~~ incremented at a time.

```
for (n=1, m=50; n<=m; n=n+1, m=m-1)
```

```
{  
    printf("\n %d %d %d", n, m, p);  
}
```

- Test Condition may have Compound relation.

```
sum=0;  
for (i=1; i<20 && sum<100; ++i)  
{  
    sum=sum+i;  
    printf("%d %d\n", sum, i);  
}
```

Testing need not be limited only to the loop control variable.

- Expressions can be used in initialization & increment / decrement sections.

```
for (x=(m+n)/2; x>0; x=x/2)
```

- One or more section can be omitted if necessary.

```
m=5;  
for (; m!=500; )  
{  
    printf("%d\n", m);  
    m=m+5;  
}
```

- For loop can be nested

```
for ( ; )  
{  
    for ( ; )  
    {  
        }  
}
```

Jumps in loops

An early exit from a loop is possible by using break or goto statement.

break statement: (i) When break statement is encountered inside a loop, the loop is immediately exited and the program continues with the statement immediately following the loop.

(ii) When the loops are nested, break would only exit from the loop containing it i.e. break will exit only a single loop. (inner loop)

e.g. while ()

```
{
```

```
    if (condn)
```

```
        break;
```

```
}
```

→ Exit from the loop

```
void main() {
```

e.g.

```
    int a, sum=0;
```

```
    while(1)
```

```
    {
```

```
        scanf ("%d", &a);
```

```
        if (a == -1)
```

```
            break;
```

```
        sum = sum + a;
```

```
    }
```

```
    printf ("%d", sum);
```

```
}
```

1st loop encounters exit from that

for ()

```
{
```

```
    for ( )
```

```
    {
```

```
        if (condn)
```

```
            break;
```

Exit from inner loop

```
    }
```

O/P

5 3 2 -1

10

Standard way of avoiding break
- use of flag variable

```

{
    int f=0;
    while(1)
    {
        if (f!=1)
            scanf("%d", &a);
        if (a== -1)
            f=1;
        sum = sum + a;
    }
    printf("%d", sum);
}

```

O/P - 5 3 2 -1

10

goto Statement

- goto statement can transfer the control to any place in a program, it is useful to provide branching within a loop.
- It is unconditionally branching statement.
- goto requires a label in order to identify the place where the branch is to be made.
- A label is any valid variable name and must be followed by a colon.

eg goto Label;

label:
Statement;

(Forward Jump)

label:
Statement;

goto label;

(Backward Jump)

- If label is before the statement goto label a loop will be formed and some statements will be executed repeatedly, such a jump is known as a backward jump.
- If label is placed after the goto label some statements will be skipped and the jump is known as a forward jump.

eg

```

void main()
{
    int n;
    scanf ("%d", &n);
    goto test;

    EVEN: printf ("In EVEN");
        goto end;
    ODD: printf ("In ODD");
        goto end;
    test:
        if (n % 2 == 0)
            goto EVEN;
        else
            goto ODD;
    end:
        printf ("n");
}

```

O/P 56
EVEN
89
ODD

```

void main()
{
    int i, num;
    i = 1;
    printf ("Enter no.");
    scanf ("%d", &num);

    table:
    printf ("%d X %d = %d",
            num, i, num * i);
    i++; if (i <= 10)
        goto table;
}

```

Continue Statement

- Continue statement tells the Compiler "SKIP THE FOLLOWING STATEMENTS AND CONTINUE WITH THE NEXT ITERATION".

In while and do-while loops Continue causes the control to go directly to the test condition and then to continue the iteration process.

In for loop, the increment section of the loop is executed before the test condition is evaluated.

eg

```

while (test Cond)
{
    if ( )
        Continue;
}

```

```

do
{
    if ( )
        Continue;
} while (cond);

```

```

for (initialization; test Cond; increment)
{
    if ( )
        Continue;
}

```

Control automatically passes to the beginning of the loop.

- Causes the next iteration of the enclosing for, while, or do loop to begin.

eg Print 1 to 5 except 3

```
void main()
```

```
{
```

```
int i;
```

```
scanf for(i=1; i<=5; i++)
```

```
{
```

```
scanf("%d", &i);
```

```
if (i == 3)
```

```
continue;
```

```
printf("%d", i);
```

```
}
```

```
}
```

o/p -

1

2

4

5

Examples

- WAP to print

```
1
1 2
1 2 3
```

n=3

```
void main()
```

```
{
```

```
int i, j, n;
```

```
printf("Enter limit");
```

```
scanf("%d", &n);
```

```
for(i=1; i<=n; i++)
```

```
{
```

```
for(j=1; j<=i; j++)
```

```
{
```

```
printf("%d", j);
```

```
}
```

```
printf("\n");
```

```
}
```

- WAP to print Floyd's triangle

```
1
2 3
4 5 6
7 8 9 10
```

```
void main()
```

```
{  
    int i, j, n, k=1;  
    printf("Enter Limit");  
    scanf("%d", &n);  
    for (i=1; i<=n; i++)  
    {  
        for (j=1; j<=i; j++, k++)  
            printf("%d", k);  
        printf("\n");  
    }  
}
```

- WAP to print sum of series $1 - \frac{x}{1!} + \frac{x^2}{2!} - \frac{x^3}{3!} + \dots - \frac{x^n}{n!}$

```
void main()
```

```
{  
    float n, i, x, fact=1, sum=1, z=1, f;  
    printf("Enter value of x & n");  
    scanf("%f %f", &x, &n);  
    for (i=1; i<=n; i++)  
    {  
        z = z * (-x);  
        fact = fact * i;  
        f = z / fact;  
        sum = sum + f;  
    }  
    printf("Sum is %f", sum);  
}
```

e.g. Enter value of ~~x~~ & n
2
2

Sum is 1.000000

- void main()

```
{  
    int n, sum=0, i;  
    scanf("%d", &n);  
    for (i=2; i<=n; i=i+2);  
        sum = sum + i;  
    printf("%d", sum); }
```

O/p - 10

- void main()

```
{  
  int a=9, b=3;  
  if (a < b || b < a)  
    printf("Hello");  
  else  
    printf("Hi");  
}
```

O/p Hello

True
non zero
(+ve, -ve)
false
0

- void main()

```
{  
  int i=5, k;  
  if (i==0)  
    goto label;  
label: printf("%d", i);  
  printf("Hey");  
}
```

O/p 5 Hey

- void main()

```
{  
  int i=0;  
  while (i==0)  
    printf("True\n");  
  printf("false\n");  
}
```

O/p - False

- void main()

```
{  
  int i=0;  
  while (i==0);  
  printf("True\n");  
  printf("False\n");  
}
```

O/p - True
False

while(i==1); then O/p is nothing

- void main()

O/p - B

```
{  
  int i=0;  
  for(;; i++)  
    printf("mA");  
    printf("mB");  
}
```

* i=1 infinite loop of A

- void main()

```
{  
  int x=1, y=5;  
  printf("%d", ++(x+y));  
}
```

O/p - Error: lvalue required

(++) operator cannot be used
with Expression

- void main()

```
{  
  int i;  
  for(i=1; i<=5; printf("in %d", i))  
    i++;  
}
```

O/p -
2
3
4
5
6